

DAY 13

1. `$@` – All Positional Parameters

`$@` represents **all positional parameters** passed to the script.

Suppose:

```
./script.sh apple banana orange
```

Then:

```
$1 = apple  
$2 = banana  
$3 = orange
```

and:

```
$@
```

represents all three:

```
apple banana orange
```

1.1 The Important Part: `"$@"`

The most important form is:

```
"$@"
```

When quoted, **each positional parameter remains a separate word/argument**.

Example:

```
./script.sh "hello world" "foo bar" test
```

The arguments are:

```
$1 = "hello world"
$2 = "foo bar"
$3 = "test"
```

Then:

```
printf '<%s>\n' "$@"
```

produces:

```
<hello world>
<foo bar>
<test>
```

So:

```
"$@"
  ↓
preserves each original argument separately
```

This is extremely important when writing scripts that forward or process arguments.

2. `$*` – All Positional Parameters as One Combined Value

`$*` also represents all positional parameters.

The major difference appears when it is **double-quoted**.

```
"$@"
```

combines the positional parameters into **one word**, using the first character of `IFS` as the separator.

If:

```
./script.sh apple banana orange
```

then conceptually:

```
"$*"
```

becomes:

```
apple banana orange
```

as **one word**.

3. `$@` vs `$*` – The Most Important Comparison

This is one of those Bash topics where the difference is subtle but very important.

Suppose:

```
./script.sh "hello world" "foo bar" test
```

```
"$@"
```

Treats each argument separately:

```
"hello world"  
"foo bar"  
"test"
```

```
"$*"
```

Combines them into one string:

```
"hello world foo bar test"
```

So the key distinction is:

```
"$@" → many arguments remain many arguments
```

```
"$*" → all arguments become one combined word
```

4. Why Quoting Matters

Consider:

```
$@
```

versus:

```
"$@"
```

The quoted version is generally what you want when you need to preserve the original arguments.

For example:

```
./script.sh "file one.txt" "file two.txt"
```

If you use:

```
printf '<%s>\n' "$@"
```

you get:

```
<file one.txt>
<file two.txt>
```

Each filename remains intact.

This is one of the most important Bash scripting habits to develop.

5. `$@` vs `$*` Cheat Table

Form	Behavior
<code>\$@</code>	All positional parameters, subject to normal unquoted expansion rules
<code>"\$@"</code>	Each positional parameter remains a separate word
<code>\$*</code>	All positional parameters, subject to normal unquoted expansion rules
<code>"\$*"</code>	All parameters combined into one word using first character of IFS

Best mental model

`"$@"` → "Give me every argument individually."

`"$*"` → "Give me all arguments together."

6. Why `$@` Is Common in Scripts

Suppose you are creating a wrapper script:

```
./wrapper.sh file1.txt file2.txt file3.txt
```

You want to pass all those arguments to another command.

You can use:

```
some_command "$@"
```

The arguments are forwarded individually.

Conceptually:

```
wrapper.sh
|
├─ file1.txt
├─ file2.txt
└─ file3.txt
```

```
↓  
some_command
```

This is extremely useful when building command wrappers and automation tools.

7. IFS and \$*

Remember:

```
IFS = Internal Field Separator
```

When "\$*" is used, Bash joins the positional parameters using the **first character of IFS**.

For example, if:

```
IFS=","
```

then:

```
"$*"
```

conceptually produces:

```
argument1,argument2,argument3
```

rather than:

```
argument1 argument2 argument3
```

This is one reason understanding IFS is important for Bash scripting.

8. read Command

Now we move from **command-line input** to **interactive input**.

The Bash `read` command allows a script to wait for input from the user.

Basic syntax:

```
read variable
```

For example:

```
read name
```

The script waits for the user to type something and press Enter.

If the user enters:

```
Sourya
```

then:

```
name = Sourya
```

9. Default `REPLY` Variable

What happens if you don't specify a variable?

For example:

```
read
```

Bash stores the input in:

```
REPLY
```

Example:

```
read  
echo "$REPLY"
```

If the user enters:

```
Hello
```

then:

```
$REPLY = Hello
```

Remember

```
read variable
  ↓
stores input in variable

read
  ↓
stores input in REPLY
```

10. `read` with Multiple Variables

You can provide multiple variables:

```
read first last
```

If the user enters:

```
Sourya Dutta
```

then:

```
first = Sourya
last  = Dutta
```

This allows a script to process multiple pieces of input.

11. What Happens with Extra Input?

Suppose:

```
read first second
```

and the user enters:

```
one two three four
```

Bash assigns the fields to the variables, with the final variable receiving the remaining input in normal `read` behavior.

Conceptually:

```
first  → one  
second → two three four
```

This behavior is useful to understand when parsing user input.

12. `read -p` – Display a Prompt

Instead of separately printing a question:

```
echo "Enter your name:"  
read name
```

you can use:

```
read -p "Enter your name: " name
```

The `-p` option means:

| Display this prompt before reading input.

Example interaction:

```
Enter your name: Sourya
```

Afterward:

```
$name = Sourya
```

13. `read -t` – Timeout

The `-t` option specifies a timeout.

Example:

```
read -t 5 name
```

This means:

| Wait up to 5 seconds for input.

If the user doesn't provide input within the specified time, `read` stops waiting.

This is useful for:

- automated scripts
- optional user interaction
- scripts that shouldn't wait forever
- security automation

14. `read -s` – Silent Input

The `-s` option prevents typed characters from being displayed on the terminal.

For example:

```
read -s -p "Password: " password
```

The user types the password, but the characters aren't echoed normally.

Conceptually:

```
Password: *****
```

instead of:

```
Password: mypassword
```

This is useful for sensitive input.

Important security note

`read -s` hides the input from normal terminal echo, but it is **not encryption**. The variable still contains the password in the shell process, so scripts should handle such data carefully.

15. Combining `read` Options

Options can be combined.

For example:

```
read -s -p "Password: " password
```

means:

```
-s → don't display typed characters  
-p → show this prompt
```

You can also use timeout:

```
read -t 10 -p "Enter value: " value
```

Meaning:

```
Prompt user  
  ↓  
Wait for input  
  ↓  
Maximum 10 seconds
```

16. `read` Cheat Sheet

Command	Purpose
<code>read name</code>	Read input into <code>name</code>
<code>read</code>	Read input into <code>REPLY</code>
<code>read a b</code>	Read into multiple variables
<code>read -p "... " name</code>	Show prompt
<code>read -t 5 name</code>	Timeout after 5 seconds
<code>read -s password</code>	Hide typed input

17. `select` Command

Now we move to something more interesting: **menu-driven scripts**.

Bash provides the `select` construct for creating a simple numbered menu.

Basic structure:

```
select variable in option1 option2 option3
do
    commands
done
```

For example:

```
select choice in Start Stop Exit
do
    ...
done
```

Bash automatically creates a menu.

Conceptually:

```
1) Start
2) Stop
3) Exit
#?
```

The user enters a number.

18. How `select` Works

Suppose:

```
select choice in Start Stop Exit
do
    echo "You selected: $choice"
done
```

Bash displays:

```
1) Start
2) Stop
3) Exit
#?
```

If the user enters:

```
1
```

then:

```
choice = Start
```

If they enter:

```
2
```

then:

```
choice = Stop
```

19. PS3

The `select` command uses a special Bash variable called:

```
PS3
```

`PS3` controls the prompt displayed by `select`.

The default prompt is generally:

```
#?
```

You can customize it:

```
PS3="Choose an option: "
```

Then:

```
1) Start
2) Stop
3) Exit
Choose an option:
```

This makes your menu much more user-friendly.

20. `select` Automatically Loops

One of the interesting properties of `select` is that it automatically repeats the menu.

For example:

```
select choice in Start Stop Exit
do
    echo "Selected: $choice"
done
```

After processing a selection, Bash displays the menu again.

So:

```
Menu
↓
User chooses
```

```
↓  
Commands execute  
↓  
Menu appears again  
↓  
User chooses  
↓  
Commands execute  
↓  
...
```

The loop continues until you explicitly leave it or terminate the script.

21. Using `break`

The normal way to exit a `select` menu from inside the loop is:

```
break
```

Example structure:

```
PS3="Choose: "  
  
select choice in Start Stop Exit  
do  
    case "$choice" in  
        Start)  
            echo "Starting..."  
            ;;  
        Stop)  
            echo "Stopping..."  
            ;;  
        Exit)  
            break  
            ;;  
    esac  
done
```

The important idea is:

```
select
  ↓
menu
  ↓
selection
  ↓
process choice
  ↓
break?
  ├──┐
  │  │
No  Yes
  │  │
  ↓  ↓
menu exit loop
again
```

22. select + case

These two constructs work very well together.

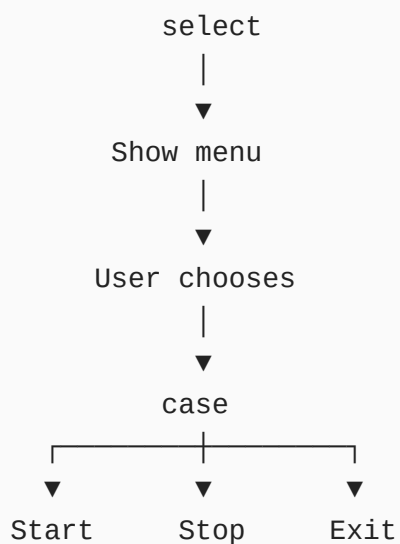
select handles:

Getting the user's menu choice

case handles:

Deciding what to do with that choice

Conceptually:



```
    |      |      |
    ▼      ▼      ▼
action  action  break
```

This pattern is worth remembering for future Bash projects.

23. Invalid Selection

Suppose the menu is:

```
1) Start
2) Stop
3) Exit
```

and the user enters:

```
5
```

There is no corresponding option.

In that case, the selection variable can be empty rather than containing a valid menu item.

You can handle that with a default `case` branch.

Conceptually:

```
*)
    echo "Invalid option"
    ;;
```

This is another reason `select` + `case` is a useful combination.

24. `select` vs `read`

These serve different purposes.

<code>read</code>	<code>select</code>
Gets arbitrary user input	Creates a numbered menu
User types text/value	User typically chooses a number
Highly flexible	Convenient for menus
You design the prompt	Bash generates the menu
Stores input in variable	Stores selected option in variable

Mental model

```
read
→ "Tell me something."

select
→ "Choose one of these."
```

25. Positional Parameters vs `read`

This distinction is also important.

Positional parameters

Input is provided **when starting the script**:

```
./script.sh file.txt
```

Inside:

```
$1 → file.txt
```

`read`

Input is provided **while the script is running**:

```
Enter filename: file.txt
```

Inside:

```
filename → file.txt
```

So:

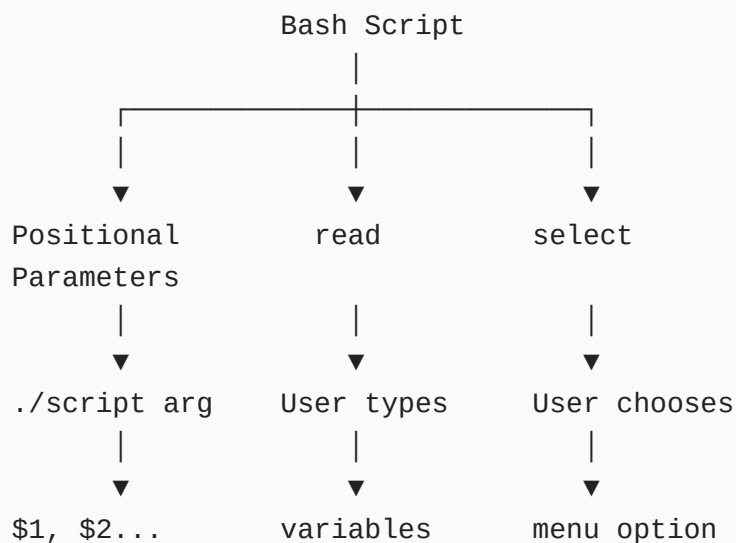
```
Command-line input → positional parameters
```

```
Interactive input → read
```

```
Menu input → select
```

26. Putting Everything Together

You now have several ways for a Bash script to receive input:



And you can validate command-line input with:

```
$#
```

and identify the script with:

```
$0
```